

EXPERIMENT-20

1. Problem Analysis:

Aim:

To develop a menu-driven modular C program that calculates geometrical and physical properties of:

Sphere

Cylinder

Cone

Cuboid

The program performs the following operations:

1. Total surface area
2. Volume
3. Base area
4. Mass
5. Weight
6. Gravitational potential energy
7. Pressure
8. Buoyant force
9. Buoyancy behaviour
10. Exit

Constants:

\[
PI=3.1416
\]

\[
GRAVITY=9.8\;m/s^2

\]

\[

TOLERANCE=0.000001

\]

SI Units Used:

Quantity	Unit
Length, radius, breadth, height	metre (m)
Area	square metre (m ²)
Volume	cubic metre (m ³)
Density	kg/m ³
Mass	kg
Weight	N
Pressure	Pa
Buoyant force	N
Potential energy	J

2. Formulae Used:

Solid/Quantity	Formula
Sphere surface area	$4\pi r^2$
Sphere volume	$\frac{4}{3}\pi r^3$
Cylinder surface area	$2\pi r(r+h)$
Cylinder volume	$\pi r^2 h$
Cone slant height	$\sqrt{r^2 + h^2}$
Cone surface area	$\pi r(l+r)$
Cone volume	$\frac{1}{3}\pi r^2 h$
Cuboid surface area	$2(lb+bh+lh)$
Cuboid volume	$l.b.h$
Circular base area	πr^2
Cuboid base area	$l.b$
Mass	$\rho.V$

Weight	$m.g$
Potential energy	$m.g.h$
Pressure	force/area
Buoyant force	$\rho_f.V.g$

For buoyancy:

solid density > fluid density → SINKS
solid density < fluid density → FLOATS
solid density = fluid density → SUSPENDED

3. Algorithm for Main Program:

Algorithm: main()

1. Start.
2. Declare variables required for menu choices and dimensions.
3. Display the main menu.
4. Input the user's choice.
5. Validate the menu choice. It must be from 1 to 10.
6. Use switch-case according to the selected operation.

7. For Surface Area:

Ask for solid type.
Input and validate required dimensions.
Call the appropriate area function.
Display area.

8. For Volume:

Ask for solid type.
Input and validate dimensions.
Call the appropriate volume function.
Display volume.

9. For Base Area:

Ask for solid type.

Allow only cylinder, cone, or cuboid.

If sphere is selected, display an error message.

Call the appropriate base-area function.

10. For Mass:

Input density.

Validate density.

Call `calculate_mass()`.

11. For Weight:

Call `calculate_weight()`.

12. For Potential Energy:

Input and validate elevation.

Call `potential_energy()`.

13. For Pressure:

Input and validate solid density.

Call `calculate_pressure()`.

14. For Buoyant Force:

Input and validate fluid density.

Call `buoyant_force()`.

15. For Buoyancy Behaviour:

Input and validate solid and fluid densities.

Call `determine_buoyancy()`.

Display FLOATS, SINKS, or SUSPENDED.

16. Repeat the menu using a do-while loop.

17. Stop when option 10 is selected.

4. Algorithms for Geometry Functions:

A. `sphere_area(radius)`

1. Start.

2. Receive radius.

3. Calculate:

```
\[  
area=4\times PI\times radius\times radius  
\]
```

4. Return area.

5. Stop.

B. `sphere_volume(radius)`

1. Start.

2. Receive radius.

3. Calculate:

```
\[  
volume=(4.0/3.0)\times PI\times radius^3  
\]
```

4. Return volume.

5. Stop.

C. cylinder_area(radius,height)

1. Start.
2. Receive radius and height.
3. Calculate:
$$\text{area} = 2 \times \text{PI} \times \text{radius} \times (\text{radius} + \text{height})$$
4. Return area.
5. Stop.

D. cylinder_volume(radius,height)

1. Start.
2. Receive radius and height.
3. Calculate:
$$\text{volume} = \text{PI} \times \text{radius}^2 \times \text{height}$$
4. Return volume.
5. Stop.

E. cone_area(radius,height)

1. Start.
2. Receive radius and height.
3. Calculate $\sqrt{\text{radius}^2 + \text{height}^2}$.
4. Pass this value to calculate_square_root().
5. Store returned value as slant height.
6. Calculate:
$$\text{area} = \text{PI} \times \text{radius} \times (\text{radius} + \text{slant_height})$$
7. Return area.
8. Stop.

F. cone_volume(radius,height)

1. Start.
2. Receive radius and height.
3. Calculate:
\[
volume=(1.0/3.0)\times \pi\times \text{radius}^2\times \text{height}
\]
4. Return volume.
5. Stop.

G. cuboid_area(length,breadth,height)

1. Start.
2. Receive length, breadth and height.
3. Calculate:
\[
area=2(lb+bh+lh)
\]
4. Return area.
5. Stop.

H. cuboid_volume(length,breadth,height)

1. Start.
2. Receive length, breadth and height.
3. Calculate $(l\times b\times h)$.
4. Return volume.
5. Stop.

I. circular_base_area(radius)

1. Start.
2. Receive radius.

3. Calculate $\pi \times \text{radius}^2$.
4. Return base area.
5. Stop.

J. cuboid_base_area(length,breadth)

1. Start.
2. Receive length and breadth.
3. Calculate $\text{length} \times \text{breadth}$.
4. Return base area.
5. Stop.

5. Algorithms for Physics Functions:

A. calculate_mass(density)

1. Start.
2. Receive density.
3. Ask for solid type.
4. Input and validate its dimensions.
5. Calculate volume by calling the corresponding volume function.
6. Calculate:


```
\[
mass=density\times volume
\]
```
7. Return mass.
8. Stop.

B. calculate_weight()

1. Start.
2. Input and validate density.
3. Call calculate_mass(density).
4. Calculate:


```
\[
```

weight=mass\times GRAVITY

\]

5. Return weight.

6. Stop.

C. potential_energy(elevation)

1. Start.

2. Receive elevation.

3. Input and validate density.

4. Obtain mass using calculate_mass().

5. Calculate:

\[

PE=mass\times GRAVITY\times elevation

\]

6. Return potential energy.

7. Stop.

D. calculate_pressure(density)

1. Start.

2. Receive density.

3. Ask for solid type.

4. Accept cylinder, cone, or cuboid.

5. Input and validate dimensions.

6. Calculate volume using the appropriate volume function.

7. Calculate contact area using the appropriate base-area function.

8. Calculate:

\[

Pressure=\frac{density\times volume\times GRAVITY}{contact\ area}

\]

9. Return pressure.

10. Stop.

E. buoyant_force(fluid_density)

1. Start.
2. Receive fluid density.
3. Ask for solid type.
4. Input and validate dimensions.
5. Calculate solid volume using the appropriate volume function.
6. Calculate:
 \[\n F_B=fluid_density\times volume\times GRAVITY\n \]
7. Return buoyant force.
8. Stop.

F. determine_buoyancy(solid_density,fluid_density)

1. Start.
2. Compare solid density with fluid density.
3. If solid density > fluid density, return SINKS.
4. Otherwise, if solid density < fluid density, return FLOATS.
5. Otherwise return SUSPENDED.
6. Stop.

6. Algorithm for Newton–Raphson Square Root

calculate_square_root(number)

1. Start.
2. Receive number.
3. If number < 0, return -1.
4. If number = 0, return 0.
5. Set:
 old_value = number
6. Start a do-while loop.

7. Calculate:

```
\[
new\_value=0.5(old\_value+\frac{number}{old\_value})
\]
```

8. Calculate:

```
\[
difference=new\_value-old\_value
\]
```

9. If difference < 0, multiply it by -1.

10. Assign:

old_value = new_value

11. Repeat while:

difference >= TOLERANCE

12. Return new_value.

13. Stop.

5. Test Case Table:

12. Test-Case Table

The question specifically requires valid, invalid and boundary inputs. A good submission should not only test ordinary calculations.

TC	Operation	Input	Type	Expected Result
1	Sphere area	r = 2	Valid	50.265600 m ²
2	Sphere volume	r = 2	Valid	33.510400 m ³
3	Sphere radius	r = 0	Invalid	Reject and ask again
4	Sphere radius	r = -2	Invalid	Reject and ask again
5	Cylinder area	r=2, h=5	Valid	87.964800 m ²
6	Cylinder volume	r=2, h=5	Valid	62.832000 m ³
7	Cylinder height	h=0	Invalid	Reject and ask again

8	Cone area	r=3, h=4	Valid	75.398400 m ²
9	Cone volume	r=3, h=4	Valid	37.699200 m ³
10	Cuboid area	l=2,b=3,h=4	Valid	52.000000 m ²
11	Cuboid volume	l=2,b=3,h=4	Valid	24.000000 m ³
12	Cylinder base	r=2	Valid	12.566400 m ²
13	Cuboid base	l=2,b=3	Valid	6.000000 m ²
14	Sphere base	Sphere	Invalid	operation Display base-area-not-defined message
15	Mass	ρ=1000, cuboid 1×1×1	Valid	1000 kg
16	Density	ρ=0	Invalid	Reject and ask again
17	Density	ρ=-100	Invalid	Reject and ask again
18	Weight	ρ=1000, cuboid 1×1×1	Valid	9800 N
19	PE	elevation=0	Valid	0 J
20	PE	elevation=-5	Invalid	Reject and ask again
21	Pressure	ρ=1000, cuboid 1×1×2	Valid	19600 Pa
22	Pressure	Sphere	Invalid	Sphere not permitted
23	Buoyant force fluid	ρ=1000, cuboid 1×1×1	(Valid)	9800 N
24	Buoyancy	solid=800, fluid=1000	(Valid)	FLOATS
25	Buoyancy	solid=1200, fluid=1000	(Valid)	SINKS
26	Buoyancy	solid=1000, fluid=1000	(Boundary)	
SUSPENDED				
27	Main menu choice=0		Invalid	Reject and ask again
28	Main menu choice=11		Invalid	Reject and ask again
29	Main menu choice=1		Valid	Accept
30	Main menu choice=10		Valid	Exit

6. Program, Compile and Execute:

```
GNU nano 9.0
#endif
#define SOLID_H

#define PI 3.1416
#define GRAVITY 9.8
#define TOLERANCE 0.000001

#define SPHERE 1
#define CYLINDER 2
#define CONE 3
#define CUBOID 4

#define FLOATS -1
#define SUSPENDED 0
#define SINKS 1

double sphere_area(double radius);
double sphere_volume(double radius);

double cylinder_area(double radius, double height);
double cylinder_volume(double radius, double height);

double cone_area(double radius, double height);
double cone_volume(double radius, double height);

double cuboid_area(double length, double breadth, double height);
double cuboid_volume(double length, double breadth, double height);

double circular_base_area(double radius);
double cuboid_base_area(double length, double breadth);

double calculate_mass(double density);
double calculate_weight();
double potential_energy(double elevation);
double calculate_pressure(double density);
double buoyant_force(double fluid_density);

int determine_buoyancy(double solid_density, double fluid_density);

double calculate_square_root(double number);

#endif
```

GNU nano 9.0

```
#include "solid.h"

double calculate_square_root(double number)
{
    double old_value, new_value, difference;

    if (number < 0)
        return -1;

    if (number == 0)
        return 0;

    old_value = number;

    do
    {
        new_value = 0.5 * (old_value + number / old_value);

        difference = new_value - old_value;

        if (difference < 0)
            difference = -difference;

        old_value = new_value;

    } while (difference ≥ TOLERANCE);

    return new_value;
}
```

GNU nano 9.0

```
#include "solid.h"

double sphere_area(double radius)
{
    return 4 * PI * radius * radius;
}

double sphere_volume(double radius)
{
    return (4.0 / 3.0) * PI * radius * radius * radius;
}

double cylinder_area(double radius, double height)
{
    return 2 * PI * radius * (radius + height);
}

double cylinder_volume(double radius, double height)
{
    return PI * radius * radius * height;
}

double cone_area(double radius, double height)
{
    double slant_height;

    slant_height = calculate_square_root(
        radius * radius + height * height
    );

    return PI * radius * (radius + slant_height);
}

double cone_volume(double radius, double height)
{
    return (1.0 / 3.0) * PI * radius * radius * height;
}
```

GNU nano 9.0

```
double cuboid_area(double length, double breadth, double height)
{
    return 2 * (length * breadth +
        breadth * height +
        length * height);
}

double cuboid_volume(double length, double breadth, double height)
{
    return length * breadth * height;
}

double circular_base_area(double radius)
{
    return PI * radius * radius;
}

double cuboid_base_area(double length, double breadth)
{
    return length * breadth;
}
```

GNU nano 9.0

```
#include <stdio.h>
#include "solid.h"

double calculate_mass(double density)
{
    int solid;
    double radius, height, length, breadth;
    double volume = 0;

    do
    {
        printf("\nSelect solid:");
        printf("\n1. Sphere");
        printf("\n2. Cylinder");
        printf("\n3. Cone");
        printf("\n4. Cuboid");
        printf("\nEnter choice: ");
        scanf("%d", &solid);

        if (solid < 1 || solid > 4)
            printf("Invalid solid choice.\n");

    } while (solid < 1 || solid > 4);

    if (solid == SPHERE)
    {
        do
        {
            printf("Enter radius: ");
            scanf("%lf", &radius);

            if (radius ≤ 0)
                printf("Radius must be greater than zero.\n");

        } while (radius ≤ 0);

        volume = sphere_volume(radius);
    }
    else if (solid == CYLINDER || solid == CONE)
    {
        do
        {
            printf("Enter radius: ");
            scanf("%lf", &radius);
```

GNU nano 9.0

```

    if (radius ≤ 0)
        printf("Radius must be greater than zero.\n");
    } while (radius ≤ 0);
    do
    {
        printf("Enter height: ");
        scanf("%lf", &height);

        if (height ≤ 0)
            printf("Height must be greater than zero.\n");
    } while (height ≤ 0);

    if (solid == CYLINDER)
        volume = cylinder_volume(radius, height);
    else
        volume = cone_volume(radius, height);
}
else
{
    do
    {
        printf("Enter length: ");
        scanf("%lf", &length);
    } while (length ≤ 0);

    do
    {
        printf("Enter breadth: ");
        scanf("%lf", &breadth);
    } while (breadth ≤ 0);

    do
    {
        printf("Enter height: ");
        scanf("%lf", &height);
    } while (height ≤ 0);

    volume = cuboid_volume(length, breadth, height);
}

return density * volume;
}
```

GNU nano 9.0

```
double calculate_weight()
{
    double density, mass;

    do
    {
        printf("Enter solid density (kg/m^3): ");
        scanf("%lf", &density);

        if (density ≤ 0)
            printf("Density must be greater than zero.\n");
    } while (density ≤ 0);

    mass = calculate_mass(density);

    return mass * GRAVITY;
}

double potential_energy(double elevation)
{
    double density, mass;

    do
    {
        printf("Enter solid density (kg/m^3): ");
        scanf("%lf", &density);

        if (density ≤ 0)
            printf("Density must be greater than zero.\n");
    } while (density ≤ 0);

    mass = calculate_mass(density);

    return mass * GRAVITY * elevation;
}

double calculate_pressure(double density)
{
    int solid;
    double radius, height, length, breadth;
    double volume, contact_area;
```

```
do
{
    printf("\nSelect solid:");
    printf("\n2. Cylinder");
    printf("\n3. Cone");
    printf("\n4. Cuboid");
    printf("\nEnter choice: ");
    scanf("%d", &solid);

    if (solid == SPHERE)
        printf("Pressure on base is not defined for a sphere.\n");
    else if (solid < CYLINDER || solid > CUBOID)
        printf("Invalid solid choice.\n");

} while (solid < CYLINDER || solid > CUBOID);

if (solid == CYLINDER || solid == CONE)
{
    do
    {
        printf("Enter radius: ");
        scanf("%lf", &radius);
    } while (radius ≤ 0);

    do
    {
        printf("Enter height: ");
        scanf("%lf", &height);
    } while (height ≤ 0);

    contact_area = circular_base_area(radius);

    if (solid == CYLINDER)
        volume = cylinder_volume(radius, height);
    else
        volume = cone_volume(radius, height);
}
else
{
    do
    {
        printf("Enter length: ");
        scanf("%lf", &length);
    } while (length ≤ 0);

    do
    {
```

```
    do
    {
        printf("Enter breadth: ");
        scanf("%lf", &breadth);
    } while (breadth ≤ 0);

    do
    {
        printf("Enter height: ");
        scanf("%lf", &height);
    } while (height ≤ 0);

    contact_area = cuboid_base_area(length, breadth);
    volume = cuboid_volume(length, breadth, height);
}

return (density * volume * GRAVITY) / contact_area;
}

double buoyant_force(double fluid_density)
{
    int solid;
    double radius, height, length, breadth;
    double volume = 0;

    do
    {
        printf("\nSelect solid:");
        printf("\n1. Sphere");
        printf("\n2. Cylinder");
        printf("\n3. Cone");
        printf("\n4. Cuboid");
        printf("\nEnter choice: ");
        scanf("%d", &solid);
    } while (solid < 1 || solid > 4);

    if (solid == SPHERE)
    {
        do
        {
            printf("Enter radius: ");
            scanf("%lf", &radius);
        } while (radius ≤ 0);

        volume = sphere_volume(radius);
    }
}
```

```
GNU nano 9.0
else if (solid == CYLINDER || solid == CONE)
{
    do
    {
        printf("Enter radius: ");
        scanf("%lf", &radius);
    } while (radius <= 0);

    do
    {
        printf("Enter height: ");
        scanf("%lf", &height);
    } while (height <= 0);

    if (solid == CYLINDER)
        volume = cylinder_volume(radius, height);
    else
        volume = cone_volume(radius, height);
}
else
{
    do
    {
        printf("Enter length: ");
        scanf("%lf", &length);
    } while (length <= 0);

    do
    {
        printf("Enter breadth: ");
        scanf("%lf", &breadth);
    } while (breadth <= 0);

    do
    {
        printf("Enter height: ");
        scanf("%lf", &height);
    } while (height <= 0);

    volume = cuboid_volume(length, breadth, height);
}

return fluid_density * volume * GRAVITY;
}
```

```
GNU nano 9.0
int determine_buoyancy(double solid_density, double fluid_density)
{
    if (solid_density > fluid_density)
        return SINKS;

    if (solid_density < fluid_density)
        return FLOATS;

    return SUSPENDED;
}
```

GNU nano 9.0

```
#include <stdio.h>
#include "solid.h"

int main()
{
    int choice, solid, result;
    double radius, height, length, breadth;
    double density, fluid_density, solid_density;
    double elevation, answer;

    do
    {
        printf("\n===== MAIN MENU =====\n");
        printf("1. Surface Area\n");
        printf("2. Volume\n");
        printf("3. Base Area\n");
        printf("4. Mass\n");
        printf("5. Weight\n");
        printf("6. Gravitational Potential Energy\n");
        printf("7. Pressure on Base\n");
        printf("8. Buoyant Force\n");
        printf("9. Buoyancy Behaviour\n");
        printf("10. Exit\n");

        do
        {
            printf("Enter choice: ");
            scanf("%d", &choice);

            if (choice < 1 || choice > 10)
                printf("Invalid menu choice. Enter 1 to 10.\n");
        } while (choice < 1 || choice > 10);

        switch (choice)
        {
            case 1:
            case 2:

                do
                {
                    printf("\n1. Sphere\n2. Cylinder\n3. Cone\n4. Cuboid\n");
                    printf("Select solid: ");
                    scanf("%d", &solid);
```

```
    if (solid < 1 || solid > 4)
        printf("Invalid solid choice.\n");
} while (solid < 1 || solid > 4);

if (solid == SPHERE)
{
    do
    {
        printf("Enter radius: ");
        scanf("%lf", &radius);

        if (radius ≤ 0)
            printf("Radius must be greater than zero.\n");

    } while (radius ≤ 0);

    if (choice == 1)
        answer = sphere_area(radius);
    else
        answer = sphere_volume(radius);
}
else if (solid == CYLINDER || solid == CONE)
{
    do
    {
        printf("Enter radius: ");
        scanf("%lf", &radius);
    } while (radius ≤ 0);

    do
    {
        printf("Enter height: ");
        scanf("%lf", &height);
    } while (height ≤ 0);

    if (solid == CYLINDER)
    {
        if (choice == 1)
            answer = cylinder_area(radius, height);
        else
            answer = cylinder_volume(radius, height);
    }
}
```

GNU nano 9.0

Home



File System



Trash

```
else
{
    if (choice == 1)
        answer = cone_area(radius, height);
    else
        answer = cone_volume(radius, height);
}
else
{
    do
    {
        printf("Enter length: ");
        scanf("%lf", &length);
    } while (length ≤ 0);

    do
    {
        printf("Enter breadth: ");
        scanf("%lf", &breadth);
    } while (breadth ≤ 0);

    do
    {
        printf("Enter height: ");
        scanf("%lf", &height);
    } while (height ≤ 0);

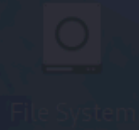
    if (choice == 1)
        answer = cuboid_area(length, breadth, height);
    else
        answer = cuboid_volume(length, breadth, height);
}

if (choice == 1)
    printf("Surface Area = %.6f square metres\n", answer);
else
    printf("Volume = %.6f cubic metres\n", answer);

break;
```

GNU nano 9.0

Home case 3:



File System



Trash

```
do
{
    printf("\n1. Sphere\n2. Cylinder\n3. Cone\n4. Cuboid\n");
    printf("Select solid: ");
    scanf("%d", &solid);
} while (solid < 1 || solid > 4);

if (solid == SPHERE)
{
    printf("Base area is not defined for a sphere.\n");
    break;
}

if (solid == CYLINDER || solid == CONE)
{
    do
    {
        printf("Enter radius: ");
        scanf("%lf", &radius);
    } while (radius ≤ 0);

    answer = circular_base_area(radius);
}
else
{
    do
    {
        printf("Enter length: ");
        scanf("%lf", &length);
    } while (length ≤ 0);

    do
    {
        printf("Enter breadth: ");
        scanf("%lf", &breadth);
    } while (breadth ≤ 0);

    answer = cuboid_base_area(length, breadth);
}

printf("Base Area = %.6f square metres\n", answer);
break;
```

case 4:

```
do
{
    printf("Enter solid density (kg/m^3): ");
    scanf("%lf", &density);

    if (density ≤ 0)
        printf("Density must be greater than zero.\n");

} while (density ≤ 0);

answer = calculate_mass(density);
printf("Mass = %.6f kg\n", answer);
break;
```

case 5:

```
answer = calculate_weight();
printf("Weight = %.6f N\n", answer);
break;
```

case 6:

```
do
{
    printf("Enter elevation (m): ");
    scanf("%lf", &elevation);

    if (elevation < 0)
        printf("Elevation cannot be negative.\n");

} while (elevation < 0);

answer = potential_energy(elevation);
printf("Potential Energy = %.6f J\n", answer);
break;
```



```

    case 7:
    do
    {
        printf("Enter solid density (kg/m^3): ");
        scanf("%lf", &density);
    } while (density ≤ 0);

    answer = calculate_pressure(density);
    printf("Pressure = %.6f Pa\n", answer);
    break;

    case 8:
    do
    {
        printf("Enter fluid density (kg/m^3): ");
        scanf("%lf", &fluid_density);
    } while (fluid_density ≤ 0);

    answer = buoyant_force(fluid_density);
    printf("Buoyant Force = %.6f N\n", answer);
    break;

    case 9:
    do
    {
        printf("Enter solid density (kg/m^3): ");
        scanf("%lf", &solid_density);
    } while (solid_density ≤ 0);

    do
    {
        printf("Enter fluid density (kg/m^3): ");
        scanf("%lf", &fluid_density);
    } while (fluid_density ≤ 0);

    result = determine_buoyancy(
        solid_density,
        fluid_density
    );

    if (result == FLOATS)
        printf("The solid FLOATS.\n");
    else if (result == SINKS)
        printf("The solid SINKS.\n");
    else
        printf("The solid remains SUSPENDED.\n");

    break;

    case 10:
        printf("Exiting program.\n");
        break;
    }

} while (choice ≠ 10);

return 0;
}

```

When we present these codes separately under the compiler, each code will run and give the desired outcome. As a result all the codes would give the right results.

NAME: TAANSIH AGARWAL

BATCH: B-15

SAP ID: 59034662